

UNIVERSIDAD AUTÓNOMA DEL ESTADO DE HIDALGO

INSTITUTO DE CIENCIAS BÁSICAS E INGENIERÍA
LICENCIATURA EN CIENCIAS COMPUTACIONALES

ASIGNATURA
Inteligencia Artificial

REPORTE DE PRÁCTICA Análisis de cadenas de símbolos

Alumno:
Daniela Lopez Martinez

Semestre:
5°

Grupo:
3

Docente:

Fecha:

Pachuca de Soto, Hidalgo

1. Introducción

Una Gramática Formal definida sobre un alfabeto Σ es una tupla de la forma:

$$G = \{\Sigma_T, \Sigma_N, S, P\}$$

donde:

- Σ_T es un alfabeto de símbolos terminales.
- Σ_N es un alfabeto de símbolos no terminales.
- S es el símbolo inicial de la gramática.
- P es un conjunto de producciones gramaticales.

Además, se cumple:

$$S \in \Sigma_N$$

$$\Sigma_T \cap \Sigma_N = \emptyset$$

$$\Sigma = \Sigma_T \cup \Sigma_N$$

La gramática formal G permite generar un lenguaje:

$$L = \{x \in \Sigma_T^* \mid S \Rightarrow^* x\}$$

Por lo tanto, las palabras del lenguaje están formadas por cadenas de símbolos terminales generadas a partir del símbolo inicial utilizando las producciones de la gramática.

Una expresión regular permite representar patrones para reconocer diferentes cadenas que pertenecen a un lenguaje regular.

En esta práctica se realiza el análisis de cadenas de símbolos para identificar números enteros, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos.

2. Marco teórico

2.1. Gramáticas formales

Una gramática formal es un sistema utilizado para definir la estructura y formación de las palabras de un lenguaje. Está formada por símbolos terminales, símbolos no terminales, un símbolo inicial y un conjunto de producciones.

En esta práctica las gramáticas formales ayudan a comprender cómo se pueden establecer reglas para reconocer diferentes cadenas.

2.2. Lenguajes formales

Un lenguaje formal es un conjunto de cadenas formadas a partir de un alfabeto. Las cadenas deben cumplir las reglas establecidas para pertenecer al lenguaje.

Los lenguajes formales son importantes en ciencias computacionales porque permiten estudiar la estructura de los lenguajes de programación.

2.3. Lenguajes regulares

Los lenguajes regulares son un tipo de lenguaje formal que puede ser representado mediante expresiones regulares y reconocido mediante autómatas finitos.

En esta práctica se utilizan para establecer patrones que permitan identificar diferentes tipos de palabras.

2.4. Expresiones regulares

Una expresión regular permite representar un patrón que deben cumplir determinadas cadenas.

Para la práctica se consideran los siguientes patrones:

Cuadro 1: Expresiones regulares utilizadas

Categoría	Expresión
Número entero	[0-9]+
Minúsculas	[a-z]+
Mayúsculas	[A-Z]+
Identificador	[a-zA-Z_] [a-zA-Z0-9_]*
Símbolo	Caracteres especiales definidos

2.5. Análisis de cadenas

El análisis de cadenas consiste en recorrer una secuencia de caracteres para saber si cumple con determinadas condiciones.

En esta práctica los programas reciben una cadena, separan sus elementos y determinan a qué categoría pertenece cada uno.

2.6. Estructuras de datos

Las estructuras de datos permiten organizar y guardar información dentro de un programa.

En el primer programa se utilizó una estructura llamada **Resultados**, que guarda los contadores y los arreglos utilizados para almacenar las palabras encontradas y sus categorías.

3. Objetivos

3.1. Objetivo general

Construir programas en un lenguaje de programación de alto nivel que permitan analizar cadenas de símbolos y clasificarlas mediante estructuras de datos, algoritmos de análisis y expresiones regulares.

3.2. Objetivos específicos

- Identificar el proceso básico de análisis de cadenas de símbolos.
- Implementar estructuras de datos y algoritmos para analizar cadenas.
- Implementar expresiones regulares para reconocer diferentes tipos de cadenas.

4. Equipo y software a utilizar

Equipo de cómputo con software para programación a elección de los alumnos.
Para el desarrollo de esta práctica se utilizó:

- Computadora.
- Lenguaje de programación C++.
- Compilador de C++.
- Entorno de desarrollo para escribir y ejecutar los programas.

5. Desarrollo

5.1. Procedimiento para el desarrollo de la práctica

1. Utilizar un lenguaje de programación para construir programas que analicen cadenas de símbolos con el propósito de identificar:

- Número entero.
- Palabras en minúsculas.
- Palabras en mayúsculas.
- Identificadores.

2. Definir el alfabeto de símbolos y las expresiones regulares para identificar:

- Números enteros.
- Palabras en minúsculas.
- Palabras en mayúsculas.

- Identificadores.
- Símbolos.

3. Construir programas que permitan clasificar y contabilizar las palabras encontradas.

5.2. Definición del alfabeto

El alfabeto considerado para la práctica está compuesto por letras mayúsculas, letras minúsculas, números y símbolos especiales.

Cuadro 2: Elementos del alfabeto

Tipo	Elementos
Letras minúsculas	a-z
Letras mayúsculas	A-Z
Números	0-9
Símbolos	+ - * / = < > ! & () { } [] ; : , . _

5.3. Reglas de identificación

Para realizar el análisis de las cadenas se utilizaron las siguientes reglas:

- Un número entero está formado únicamente por dígitos.
- Una palabra en minúsculas está formada por letras de la “a” a la “z”.
- Una palabra en mayúsculas está formada por letras de la “A” a la “Z”.
- Un identificador comienza con una letra o un guion bajo y después puede contener letras, números o guiones bajos.
- Un símbolo corresponde a uno de los caracteres especiales definidos dentro del alfabeto.

5.4. Programa 1: análisis mediante estructuras de datos

El primer programa utiliza una estructura llamada **Resultados**. Esta estructura permite almacenar los contadores y las palabras encontradas.

El programa solicita una cadena al usuario y analiza cada elemento para determinar si corresponde a un número, palabra en minúsculas, palabra en mayúsculas, identificador o símbolo.

5.4.1. Código del programa 1

```

1 #include <iostream>
2 #include <cstring>
3 #include <cctype>
4 using namespace std;
5
6 struct Resultados {
7     int numerosEnteros;
```

```

8     int palabrasMinusculas;
9     int palabrasMayusculas;
10    int identificadores;
11    int simbolos;
12    char palabras[100][50];
13    char tipos[100][20];
14    int contadorPalabras;
15 };
16
17 bool esNumeroEntero(char palabra[]) {
18     int len = strlen(palabra);
19
20     if (len == 0) {
21         return false;
22     }
23
24     for (int i = 0; i < len; i++) {
25         if (!isdigit(palabra[i])) {
26             return false;
27         }
28     }
29
30     return true;
31 }
32
33 bool esPalabraMinuscula(char palabra[]) {
34     int len = strlen(palabra);
35
36     if (len == 0) {
37         return false;
38     }
39
40     for (int i = 0; i < len; i++) {
41         if (!islower(palabra[i])) {
42             return false;
43         }
44     }
45
46     return true;
47 }
48
49 bool esPalabraMayuscula(char palabra[]) {
50     int len = strlen(palabra);
51
52     if (len == 0) {
53         return false;
54     }
55
56     for (int i = 0; i < len; i++) {
57         if (!isupper(palabra[i])) {
58             return false;
59         }
60     }
61
62     return true;
63 }
64
65 bool esIdentificador(char palabra[]) {
66     int len = strlen(palabra);
67
68     if (len == 0) {
69         return false;
70     }
71
72     if (!isalpha(palabra[0]) && palabra[0] != '_' ) {
73         return false;
74     }
75
76     for (int i = 1; i < len; i++) {
77         if (!isalnum(palabra[i]) && palabra[i] != '_' ) {
78             return false;
79         }
80     }

```

```

81
82     return true;
83 }
84
85 bool esSimbolo(char palabra[]) {
86     char simbolos[] =
87         "+-*/=<>!&|(){}[];:,. _'\\"";
88
89     int len = strlen(palabra);
90
91     if (len == 0) {
92         return false;
93     }
94
95     for (int i = 0; i < len; i++) {
96
97         bool encontrado = false;
98
99         for (int j = 0; j < strlen(simbolos); j++) {
100
101             if (palabra[i] == simbolos[j]) {
102                 encontrado = true;
103                 break;
104             }
105         }
106
107         if (!encontrado) {
108             return false;
109         }
110     }
111 }
112
113     return true;
114 }
115
116 void clasificarPalabra(char palabra[],
117                         Resultados &resultados) {
118
119     if (esNumeroEntero(palabra)) {
120
121         resultados.numerosEnteros++;
122
123         strcpy(resultados.palabras[
124             resultados.contadorPalabras], palabra);
125
126         strcpy(resultados.tipos[
127             resultados.contadorPalabras],
128             "NUMERO_ENTERO");
129
130         resultados.contadorPalabras++;
131     }
132
133     else if (esPalabraMinuscula(palabra)) {
134
135         resultados.palabrasMinusculas++;
136
137         strcpy(resultados.palabras[
138             resultados.contadorPalabras], palabra);
139
140         strcpy(resultados.tipos[
141             resultados.contadorPalabras],
142             "PALABRA_MINUSCULA");
143
144         resultados.contadorPalabras++;
145     }
146
147     else if (esPalabraMayuscula(palabra)) {
148
149         resultados.palabrasMayusculas++;
150
151         strcpy(resultados.palabras[
152             resultados.contadorPalabras], palabra);
153

```

```

154         strcpy(resultados.tipos[
155             resultados.contadorPalabras],
156             "PALABRA_MAYUSCULA");
157
158         resultados.contadorPalabras++;
159     }
160
161     else if (esIdentificador(palabra)) {
162
163         resultados.identificadores++;
164
165         strcpy(resultados.palabras[
166             resultados.contadorPalabras], palabra);
167
168         strcpy(resultados.tipos[
169             resultados.contadorPalabras],
170             "IDENTIFICADOR");
171
172         resultados.contadorPalabras++;
173     }
174
175     else if (esSimbolo(palabra)) {
176
177         resultados.simbolos++;
178
179         strcpy(resultados.palabras[
180             resultados.contadorPalabras], palabra);
181
182         strcpy(resultados.tipos[
183             resultados.contadorPalabras],
184             "SIMBOLO");
185
186         resultados.contadorPalabras++;
187     }
188 }
189
190 void analizarCadena(char cadena[],
191                     Resultados &resultados) {
192
193     int len = strlen(cadena);
194     char palabra[50];
195     int pos = 0;
196
197     for (int i = 0; i <= len; i++) {
198
199         if (cadena[i] == ' ' ||
200             cadena[i] == '\t' ||
201             cadena[i] == '\n' ||
202             cadena[i] == '\0') {
203
204             if (pos > 0) {
205
206                 palabra[pos] = '\0';
207
208                 clasificarPalabra(
209                     palabra,
210                     resultados
211                 );
212
213                 pos = 0;
214             }
215
216             } else {
217
218                 palabra[pos++] = cadena[i];
219             }
220         }
221     }
222
223 void mostrarResultados(Resultados &resultados) {
224
225     cout << "\n=== Analisis ===" << endl;
226

```

```

227     cout << "Numeros enteros: "
228         << resultados.numerosEnteros
229         << endl;
230
231     cout << "Palabras en minusculas: "
232         << resultados.palabrasMinusculas
233         << endl;
234
235     cout << "Palabras en mayusculas: "
236         << resultados.palabrasMayusculas
237         << endl;
238
239     cout << "Identificadores: "
240         << resultados.identificadores
241         << endl;
242
243     cout << "Simbolos: "
244         << resultados.simbolos
245         << endl;
246
247     cout << "\n=== DETALLE DE PALABRAS ENCONTRADAS ==="
248         << endl;
249
250     for (int i = 0;
251         i < resultados.contadorPalabras;
252         i++) {
253
254         cout << "Palabra: "
255             << resultados.palabras[i]
256             << " -> Tipo: "
257             << resultados.tipos[i]
258             << endl;
259     }
260 }
261
262 int main() {
263
264     Resultados resultados;
265
266     resultados.numerosEnteros = 0;
267     resultados.palabrasMinusculas = 0;
268     resultados.palabrasMayusculas = 0;
269     resultados.identificadores = 0;
270     resultados.simbolos = 0;
271     resultados.contadorPalabras = 0;
272
273     cout << "\n=== ANALISIS DE CADENA PERSONALIZADA ==="
274         << endl;
275
276     char entrada[500];
277
278     cout << "Ingrese una cadena para analizar: ";
279
280     cin.getline(entrada, 500);
281
282     analizarCadena(entrada, resultados);
283
284     mostrarResultados(resultados);
285
286     return 0;
287 }

```

5.4.2. Funcionamiento del programa 1

El programa realiza las siguientes actividades:

1. Declara la estructura `Resultados`.
2. Inicializa los contadores.

3. Solicita una cadena al usuario.
4. Separa la cadena en palabras.
5. Revisa cada palabra.
6. Clasifica las palabras.
7. Actualiza los contadores.
8. Muestra los resultados.

5.5. Programa 2: identificación del alfabeto y clasificación

El segundo programa muestra el alfabeto utilizado y los patrones establecidos para cada categoría. También permite introducir una cadena y clasificar sus elementos.

5.5.1. Código del programa 2

```
1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4
5  bool esNumero(char palabra[]) {
6
7      int len = strlen(palabra);
8
9      if (len == 0) {
10         return false;
11     }
12
13     for(int i = 0; i < len; i++) {
14
15         if(palabra[i] < '0' ||
16            palabra[i] > '9') {
17
18             return false;
19         }
20     }
21
22     return true;
23 }
24
25 bool esMinuscula(char palabra[]) {
26
27     int len = strlen(palabra);
28
29     if (len == 0) {
30         return false;
31     }
32
33     for(int i = 0; i < len; i++) {
34
35         if(palabra[i] < 'a' ||
36            palabra[i] > 'z') {
37
38             return false;
39         }
40     }
41
42     return true;
43 }
44
45 bool esMayuscula(char palabra[]) {
46
```

```

47     int len = strlen(palabra);
48
49     if (len == 0) {
50         return false;
51     }
52
53     for(int i = 0; i < len; i++) {
54
55         if(palabra[i] < 'A' ||
56            palabra[i] > 'Z') {
57
58             return false;
59         }
60     }
61
62     return true;
63 }
64
65 bool esIdentificador(char palabra[]) {
66
67     int len = strlen(palabra);
68
69     if (len == 0) {
70         return false;
71     }
72
73     char primera = palabra[0];
74
75     if(!(primera >= 'a' &&
76         primera <= 'z') &&
77        !(primera >= 'A' &&
78         primera <= 'Z') &&
79        primera != '_' ) {
80
81         return false;
82     }
83
84     for(int i = 1; i < len; i++) {
85
86         char c = palabra[i];
87
88         if(!(c >= 'a' && c <= 'z') &&
89            !(c >= 'A' && c <= 'Z') &&
90            !(c >= '0' && c <= '9') &&
91            c != '_' ) {
92
93             return false;
94         }
95     }
96
97     return true;
98 }
99
100 bool esSimbolo(char palabra[]) {
101
102     char simbolos[] =
103         "+-*/=<>|&|(){}[];:,._'\\"";
104
105     int len = strlen(palabra);
106
107     if (len == 0) {
108         return false;
109     }
110
111     for(int i = 0; i < len; i++) {
112
113         bool encontrado = false;
114
115         for(int j = 0;
116             j < strlen(simbolos);
117             j++) {
118
119             if(palabra[i] == simbolos[j]) {

```

```

120
121         encontrado = true;
122         break;
123     }
124 }
125
126     if(!encontrado) {
127         return false;
128     }
129 }
130
131     return true;
132 }
133
134 void clasificar(char palabra[],
135                 char tipo[]) {
136
137     if(esNumero(palabra)) {
138
139         strcpy(tipo, "NUMERO");
140     }
141
142     else if(esMinuscula(palabra)) {
143
144         strcpy(tipo, "MINUSCULA");
145     }
146
147     else if(esMayuscula(palabra)) {
148
149         strcpy(tipo, "MAYUSCULA");
150     }
151
152     else if(esIdentificador(palabra)) {
153
154         strcpy(tipo, "IDENTIFICADOR");
155     }
156
157     else if(esSimbolo(palabra)) {
158
159         strcpy(tipo, "SIMBOLO");
160     }
161
162     else {
163
164         strcpy(tipo, "DESCONOCIDO");
165     }
166 }
167
168 int main() {
169
170     cout << "=====
171         << endl;
172
173     cout << "ALFABETO Y EXPRESIONES REGULARES"
174         << endl;
175
176     cout << "=====
177         << endl;
178
179     cout << "\nALFABETO:" << endl;
180
181     cout << "Letras: a-z, A-Z" << endl;
182
183     cout << "Numeros: 0-9" << endl;
184
185     cout << "Simbolos: + - * / = < > ! & | "
186         "( ) { } [ ] ; : , . _"
187         << endl;
188
189     cout << "\nEXPRESIONES REGULARES:"
190         << endl;
191
192     cout << "1. NUMERO: solo digitos (0-9)"

```

```

193         << endl;
194
195     cout << "2. MINUSCULA: solo letras a-z"
196         << endl;
197
198     cout << "3. MAYUSCULA: solo letras A-Z"
199         << endl;
200
201     cout << "4. IDENTIFICADOR: empieza con letra "
202         "o _, luego letras, numeros o _"
203         << endl;
204
205     cout << "5. SIMBOLO: caracteres especiales"
206         << endl;
207
208     cout << "\n===== "
209         << endl;
210
211     cout << "ANALISIS PERSONALIZADO"
212         << endl;
213
214     cout << "===== "
215         << endl;
216
217     char palabras[50][50];
218
219     char tipos[50][30];
220
221     char entrada[500];
222
223     char palabra[50];
224
225     int pos = 0;
226
227     int contNum = 0;
228     int contMin = 0;
229     int contMay = 0;
230     int contId = 0;
231     int contSim = 0;
232
233     int totalPalabras = 0;
234
235     cout << "Escribe una frase para analizar: ";
236
237     cin.getline(entrada, 500);
238
239     for(int i = 0;
240         i <= strlen(entrada);
241         i++) {
242
243         if(entrada[i] == ' ' ||
244             entrada[i] == '\0') {
245
246             if(pos > 0) {
247
248                 palabra[pos] = '\0';
249
250                 clasificar(
251                     palabra,
252                     tipos[totalPalabras]
253                 );
254
255                 strcpy(
256                     palabras[totalPalabras],
257                     palabra
258                 );
259
260                 if(strcmp(
261                     tipos[totalPalabras],
262                     "NUMERO") == 0)
263
264                     contNum++;
265

```

```

266         else if(strcmp(
267             tipos[totalPalabras],
268             "MINUSCULA") == 0)
269
270             contMin++;
271
272         else if(strcmp(
273             tipos[totalPalabras],
274             "MAYUSCULA") == 0)
275
276             contMay++;
277
278         else if(strcmp(
279             tipos[totalPalabras],
280             "IDENTIFICADOR") == 0)
281
282             contId++;
283
284         else if(strcmp(
285             tipos[totalPalabras],
286             "SIMBOLO") == 0)
287
288             contSim++;
289
290         totalPalabras++;
291
292         pos = 0;
293     }
294
295     } else {
296
297         palabra[pos] = entrada[i];
298
299         pos++;
300     }
301 }
302
303 cout << "\nResultados:" << endl;
304
305 for(int i = 0;
306     i < totalPalabras;
307     i++) {
308
309     cout << (i+1)
310         << ". "
311         << palabras[i]
312         << " => "
313         << tipos[i]
314         << endl;
315 }
316
317 cout << "\nEstadisticas:"
318     << endl;
319
320 cout << "Numeros: "
321     << contNum
322     << endl;
323
324 cout << "Minusculas: "
325     << contMin
326     << endl;
327
328 cout << "Mayusculas: "
329     << contMay
330     << endl;
331
332 cout << "Identificadores: "
333     << contId
334     << endl;
335
336 cout << "Simbolos: "
337     << contSim
338     << endl;

```

```

339
340     cout << "Total: "
341           << totalPalabras
342           << endl;
343
344     cout << "\nFIN DEL PROGRAMA"
345           << endl;
346
347     return 0;
348 }

```

5.6. Programa 3: análisis mediante expresiones regulares

El tercer programa realiza el análisis de cadenas utilizando directamente expresiones regulares de C++. Para esto se utiliza la biblioteca `<regex>`, que permite definir patrones para reconocer números, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos.

A diferencia de los programas anteriores, en este programa las reglas de reconocimiento se escriben directamente mediante expresiones regulares y se utilizan con la función `regex_match()`.

5.6.1. Código del programa 3

```

1  #include <iostream>
2  #include <regex>
3  #include <string>
4  using namespace std;
5
6  string clasificar(string palabra) {
7      regex numero("[0-9]+$");
8      regex minuscula("[a-z]+$");
9      regex mayuscula("[A-Z]+$");
10     regex identificador("[a-zA-Z_][a-zA-Z0-9_]*$");
11     regex simbolo("[+\\-*/=<>|&|(){}\\[\\] ;:,.']+$");
12
13     if(regex_match(palabra, numero))
14         return "NUMERO";
15
16     if(regex_match(palabra, minuscula))
17         return "MINUSCULA";
18
19     if(regex_match(palabra, mayuscula))
20         return "MAYUSCULA";
21
22     if(regex_match(palabra, identificador))
23         return "IDENTIFICADOR";
24
25     if(regex_match(palabra, simbolo))
26         return "SIMBOLO";
27
28     return "DESCONOCIDO";
29 }
30
31 int main() {
32
33     cout << "ANALIZADOR DE PALABRAS" << endl;
34
35     cout << "\nEXPRESIONES REGULARES:" << endl;
36
37     cout << "1. NUMERO: [0-9]+$ (ej: 123, 45)" << endl;
38
39     cout << "2. MINUSCULA: [a-z]+$ (ej: hola, mundo)"
40         << endl;
41
42     cout << "3. MAYUSCULA: [A-Z]+$ (ej: HOLA, MUNDO)"
43         << endl;

```

```

44
45 cout << "4. IDENTIFICADOR: "
46     "[a-zA-Z_][a-zA-Z0-9_]*$ "
47     "(ej: var1, _temp)"
48     << endl;
49
50 cout << "5. SIMBOLO: "
51     "[+\\-*/=<>!&|(){}\\[\\] ;:.,']+$ "
52     "(ej: +, -, =)"
53     << endl;
54
55 cout << "Ingresa una frase para analizar: ";
56
57 string entrada;
58
59 getline(cin, entrada);
60
61 string palabras[100];
62
63 string tipos[100];
64
65 int total = 0;
66
67 int cNum = 0;
68 int cMin = 0;
69 int cMay = 0;
70 int cId = 0;
71 int cSim = 0;
72
73 string palabra = "";
74
75 for(int i = 0;
76     i <= entrada.length();
77     i++) {
78
79     if(i == entrada.length() ||
80         entrada[i] == ' ') {
81
82         if(palabra != "") {
83
84             string tipo = clasificar(palabra);
85
86             palabras[total] = palabra;
87
88             tipos[total] = tipo;
89
90             total++;
91
92             if(tipo == "NUMERO")
93                 cNum++;
94
95             else if(tipo == "MINUSCULA")
96                 cMin++;
97
98             else if(tipo == "MAYUSCULA")
99                 cMay++;
100
101             else if(tipo == "IDENTIFICADOR")
102                 cId++;
103
104             else if(tipo == "SIMBOLO")
105                 cSim++;
106
107             palabra = "";
108         }
109
110     } else {
111
112         palabra += entrada[i];
113     }
114 }
115
116 cout << "RESULTADOS" << endl;

```

```

117
118     cout << "\nPalabras encontradas:" << endl;
119
120     for(int i = 0; i < total; i++) {
121
122         cout << (i+1)
123             << ". '"
124             << palabras[i]
125             << "' = "
126             << tipos[i]
127             << endl;
128     }
129
130     cout << "\nEstadísticas:" << endl;
131
132     cout << "Numeros: "
133         << cNum
134         << endl;
135
136     cout << "Minúsculas: "
137         << cMin
138         << endl;
139
140     cout << "Mayúsculas: "
141         << cMay
142         << endl;
143
144     cout << "Identificadores: "
145         << cId
146         << endl;
147
148     cout << "Símbolos: "
149         << cSim
150         << endl;
151
152     cout << "Total palabras: "
153         << total
154         << endl;
155
156     return 0;
157 }

```

5.6.2. Funcionamiento del programa 3

El programa realiza el análisis de la cadena mediante expresiones regulares. Su funcionamiento se puede resumir de la siguiente manera:

1. Importa las bibliotecas `regex` y `string`.
2. Define una expresión regular para cada categoría.
3. Recibe una frase escrita por el usuario.
4. Separa la frase en palabras.
5. Utiliza la función `clasificar()` para analizar cada palabra.
6. La función `regex_match()` comprueba si la palabra cumple con alguno de los patrones.
7. Guarda cada palabra y su clasificación.
8. Cuenta cuántos elementos pertenecen a cada categoría.
9. Finalmente muestra las palabras encontradas y las estadísticas.

5.6.3. Expresiones regulares utilizadas en el programa 3

Cuadro 3: Expresiones regulares del programa 3

Categoría	Expresión regular
Número	<code>^[0-9]+\$</code>
Minúscula	<code>^[a-z]+\$</code>
Mayúscula	<code>^[A-Z]+\$</code>
Identificador	<code>^[a-zA-Z_][a-zA-Z0-9_]*\$</code>
Símbolo	Caracteres especiales permitidos

5.6.4. Análisis del programa 3

Este programa permite observar de una manera más directa el uso de expresiones regulares para el reconocimiento de cadenas.

Por ejemplo, la expresión `^[0-9]+$` permite reconocer una cadena formada únicamente por números. De manera similar, `^[a-z]+$` reconoce palabras formadas solamente por letras minúsculas.

La expresión utilizada para los identificadores permite que la primera posición sea una letra o un guion bajo y que las posiciones siguientes puedan contener letras, números o guiones bajos.

El uso de `regex_match()` facilita la comparación de cada palabra con los patrones establecidos y permite clasificar automáticamente los elementos de la cadena.

6. Resultados

Los programas desarrollados permitieron analizar cadenas introducidas por el usuario y clasificar los elementos de acuerdo con las categorías establecidas.

Los resultados obtenidos corresponden a:

- Números enteros.
- Palabras en minúsculas.
- Palabras en mayúsculas.
- Identificadores.
- Símbolos.

6.1. Resultado del programa 1

En esta sección se colocará una captura de pantalla donde se observe la ejecución del primer programa.

6.4. Análisis de los resultados

Los programas permitieron reconocer diferentes tipos de cadenas de acuerdo con las reglas establecidas.

El primer programa utiliza estructuras y funciones para realizar la clasificación.

El segundo programa muestra el alfabeto y los patrones utilizados para reconocer cada categoría.

El tercer programa utiliza directamente expresiones regulares mediante la biblioteca `regex` de C++. Esto permite realizar la comparación de las palabras con los patrones establecidos de una manera más directa.

Con las pruebas realizadas se pudo comprobar que las reglas permiten identificar y clasificar las cadenas introducidas por el usuario.

7. Cuestionario

7.1. ¿Cuál es la utilidad de las expresiones regulares en la identificación de palabras de un lenguaje?

Sirven para reconocer palabras o cadenas que cumplen ciertas reglas. En la práctica las usamos para identificar números, palabras e identificadores.

7.2. ¿Cuáles elementos del lenguaje de programación utilizaste para la implementación de las estructuras de datos?, ¿Cómo las utilizaste?

Utilicé estructuras, arreglos, funciones, ciclos y condiciones. Las estructuras y arreglos sirvieron para guardar los resultados y las funciones ayudaron a analizar las palabras.

7.3. ¿Cuál es el proceso que seguiste para analizar las cadenas de símbolos?

Primero se pide una cadena al usuario. Después se separa en palabras y se revisa cada una para saber qué tipo es. Al final se muestran los resultados.

7.4. ¿Qué elementos del lenguaje de programación te permitieron implementar expresiones regulares?, ¿qué condiciones o recomendaciones se deben seguir al escribir las expresiones regulares?

Utilicé rangos de caracteres como `[0-9]`, `[a-z]` y `[A-Z]` para definir los patrones. En el tercer programa utilicé la biblioteca `regex` y la función `regex_match()`.

Se deben establecer correctamente los caracteres permitidos y los límites del patrón para que el programa clasifique correctamente las palabras.

8. Conclusiones

Con esta práctica pude comprender mejor cómo se pueden analizar cadenas de símbolos utilizando reglas y patrones.

También aprendí a utilizar estructuras, arreglos, funciones y condiciones en C++ para clasificar diferentes tipos de palabras.

Los programas permitieron identificar números, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos.

Además, con el tercer programa pude utilizar expresiones regulares directamente mediante la biblioteca `regex`, lo que permitió comparar las cadenas con diferentes patrones de una manera más sencilla.

Finalmente, la práctica ayudó a relacionar los conceptos de lenguajes formales y expresiones regulares con programas que realizan el análisis y clasificación de cadenas.

9. Bibliografía

1. Giró, J., Vázquez, J., Meloni, B., & Constable, L. (2015). *Lenguajes formales y teoría de autómatas*. Editorial Alfaomega. Argentina.
2. Ruiz Catalán, J. (2010). *Compiladores: teoría e implementación*. Editorial Alfaomega. México.
3. Brookshear, J. G. (1995). *Teoría de la Computación. Lenguajes formales, autómatas y complejidad*. Editorial Addison-Wesley Iberoamericana. Primera edición. U.S.A.